MNIST Digit Classification Project Report

Manual Implementation of MLP and CNN Models

Linn Erle Kloefta

Lklfta1@student.gsu.edu

CSC4851 Spring 2026

April 20. 2026

Contents

1 Introduction **2**
- 1.1 Background and Motivation . . . 2
- 1.2 Problem Definition . . . 2
- 1.3 System Overview . . . 2

2 Methodology **2**
- 2.1 Implementation Setup . . . 2
- 2.2 Data Pipeline and Preprocessing . . . 2
  - 2.2.1 Multilayer Perceptron (MLP) Implementation . . . 3
  - 2.2.2 Convolutional Neural Network (CNN) Implementation . . . 3
- 2.3 Concrete Training Walkthrough . . . 4

3 Evaluation **4**
- 3.1 Experimental Settings . . . 4
- 3.2 Training Progression and Feature Extraction . . . 4
- 3.3 Qualitative Profiling and Confusion Analysis . . . 6

4 Learning Outcomes **7**
- 4.1 Tasks Accomplished . . . 7
- 4.2 Challenges and Debugging Realities . . . 7
- 4.3 Lessons Learned . . . 8

1 Introduction

1.1 Background and Motivation

MNIST digit classification is a common way to test how neural networks learn. Since this was an individual project, I used it as a chance to understand that learning process myself. The assignment required manual implementation, so instead of using high-level deep-learning layers, I implemented forward propagation, loss calculation, and backpropagation directly using NumPy.

1.2 Problem Definition

The main task is to classify 28×28 grayscale images of digits (0–9). In this report, I compare the two required architectures based on how they train, how well they perform on test data, and where they still make mistakes.

1.3 System Overview

I study two manually implemented models:

1. **Multilayer Perceptron (MLP):** a 2-layer fully connected network with Sigmoid in the hidden layer and Softmax in the output layer, trained for up to 30 epochs.
2. **Convolutional Neural Network (CNN):** one valid-mode convolution layer with ReLU, followed by a fully connected Softmax output layer, trained for up to 5 epochs.

For both models, I use the same MNIST train/test split and evaluate them using training logs and the figures included in this report.

2 Methodology

2.1 Implementation Setup

I implemented the two models in separate files: `MLP_template.py` for the fully connected network and `CNN_template.py` for the convolutional network. In both files, the training logic is written manually: forward pass, loss calculation, backward pass, and parameter update.

2.2 Data Pipeline and Preprocessing

The same data pipeline is used in both templates, and it is implemented in three steps:

1. Load MNIST from `./data/mnist` using torchvision dataloaders.
2. Convert images with `ToTensor()`.
3. Normalize with `Normalize((0.5,), (0.5,))` so inputs are centered around zero.

Centering inputs around zero helps keep activations and gradients more stable during training.

Batch size is model-specific in the templates:

- **MLP batch size = 128:** gives efficient matrix updates and smoother gradients from batch averaging.
- **CNN batch size = 64:** keeps training stable while limiting the cost of manual convolution loops in NumPy.

2.2.1 Multilayer Perceptron (MLP) Implementation

The MLP implementation in `MLP_template.py` follows this step-by-step flow:

1. Flatten each 28×28 image into a 784-dimensional vector.
2. Run the first linear layer and Sigmoid activation.
3. Run the second linear layer and Softmax to get class probabilities.
4. Compute cross-entropy loss, backpropagate gradients, and update weights with gradient descent.

The forward equations are:

$$Z^{[1]} = XW^{[1]} + B^{[1]}, \quad A^{[1]} = \sigma\left(Z^{[1]}\right) = \frac{1}{1 + e^{-Z^{[1]}}} \tag{1}$$

$$Z^{[2]} = A^{[1]}W^{[2]} + B^{[2]}, \quad A^{[2]} = \text{softmax}\left(Z^{[2]}\right) \tag{2}$$

Backpropagation uses Softmax–cross-entropy output gradients and Sigmoid chain-rule terms. The chosen MLP parameters are:

- **Input size** 784**:** required by flattening 28×28 images.
- **Hidden size** 128**:** gives enough model capacity without adding too much extra cost; Figure 7 shows only small gains beyond this point.
- **Output size** 10**:** fixed by MNIST class count.
- **Learning rate** 0.5**:** chosen so the MLP learns quickly in early epochs with Sigmoid activation.
- **Weight init scale** 0.05**:** avoids very large pre-activation values that can saturate Sigmoid.
- **Loss epsilon** 10^{-15}**:** prevents invalid $\log(0)$ in cross-entropy.
- **Epochs** 30**:** assignment-compliant training budget and sufficient for convergence.

2.2.2 Convolutional Neural Network (CNN) Implementation

The CNN implementation in `CNN_template.py` also follows a manual step-by-step flow:

1. Keep input images in 2D form and apply one valid-mode convolution filter.
2. Apply ReLU to the convolution output.
3. Flatten the feature map and apply a fully connected Softmax classifier.
4. Compute cross-entropy loss, backpropagate through dense and convolution layers, and update parameters.

This gives the following structure:

- **Convolution stage:** one filter with kernel size 7 and ReLU nonlinearity.
- **Classification stage:** flattened feature map projected to 10 classes with Softmax output.

The chosen CNN parameters are:

- **Input size** 28**:** keeps the spatial image layout explicit for valid-mode convolution.

- **One convolution kernel:** matches the required single-convolution-layer architecture and keeps implementation manageable.
- **Kernel size** 7**:** selected from tested candidates as the best-performing setting under the fixed 5-epoch budget (Figure 6).
- **Flattened size** 484**:** follows directly from valid convolution, $(28 - 7 + 1)^2 = 22^2$.
- **Output size** 10**:** fixed by MNIST class count.
- **Learning rate** 0.05**:** lower than MLP to keep CNN updates stable with manual convolution and dense projection.
- **Loss epsilon** 10^{-8}**:** prevents invalid logarithms in CNN cross-entropy.
- **Epochs** 5**:** assignment-compliant training budget.

2.3 Concrete Training Walkthrough

Figure 1 shows one failed-guess case from my manual training pipeline. The sequence is forward pass, cross-entropy loss, backward pass, and parameter update. I include this case because it clearly shows how a wrong prediction produces a stronger correction signal, which then pushes probability toward the true class in the next update.

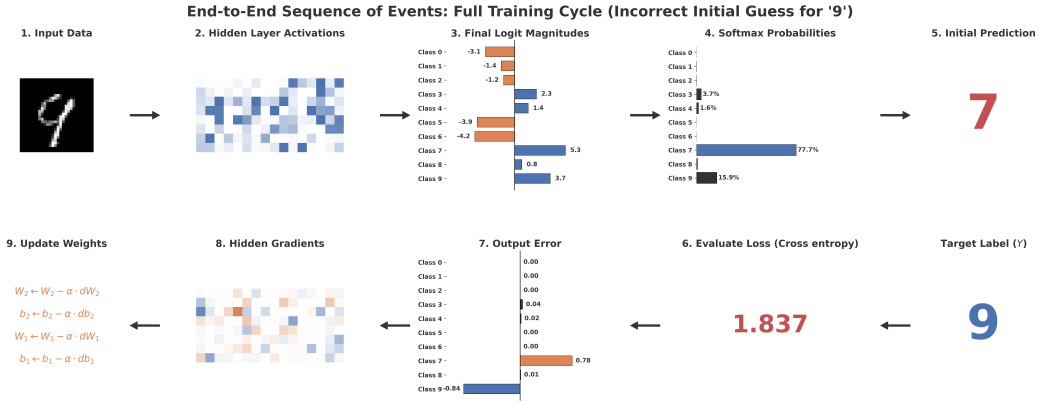

Figure 1: Manual MLP training flow for a failed guess and correction.

3 Evaluation

3.1 Experimental Settings

Experiments follow the exact training loops in the template files. MLP is trained with hidden size 128, learning rate 0.5, batch size 128, and 30 epochs; CNN uses one 7×7 valid convolution kernel, learning rate 0.05, batch size 64, and 5 epochs.

3.2 Training Progression and Feature Extraction

From Figure 3, I see that the MLP improves quickly at the start and then keeps improving more slowly across later epochs. In this run, test accuracy rises from 0.9166 (epoch 1) to 0.9790 (epoch 30).

Figure 5 shows a different behavior for CNN. Training loss goes down in all 5 epochs, but test accuracy goes up and down, with a dip at epoch 4 (0.8803) before ending at 0.9178. To

me, this suggests unstable short-run behavior rather than steady generalization from epoch to epoch.

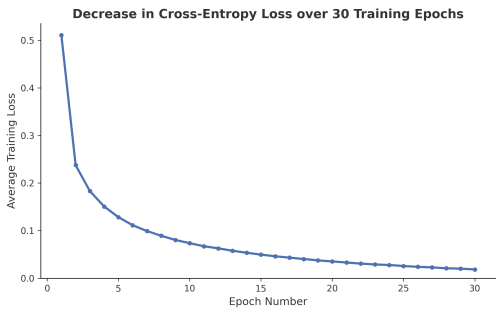

Figure 2: MLP training loss curve.

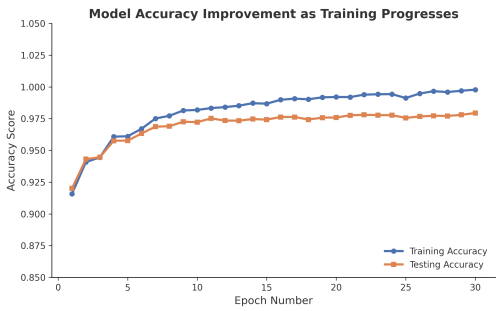

Figure 3: MLP train/test accuracy curve.

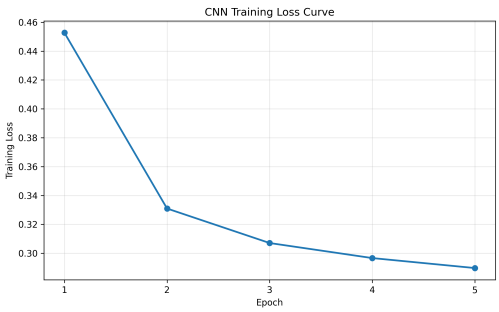

Figure 4: CNN training loss curve.

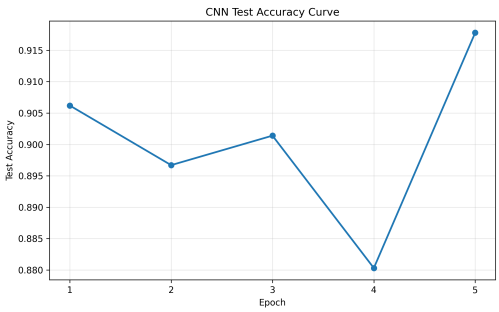

Figure 5: CNN test accuracy curve.

Figure 6 summarizes kernel-size behavior. Among the tested settings, 7×7 gives the highest final test accuracy (0.9311) and still keeps loss at a competitive level, so it is the most reasonable choice under the fixed 5-epoch budget.

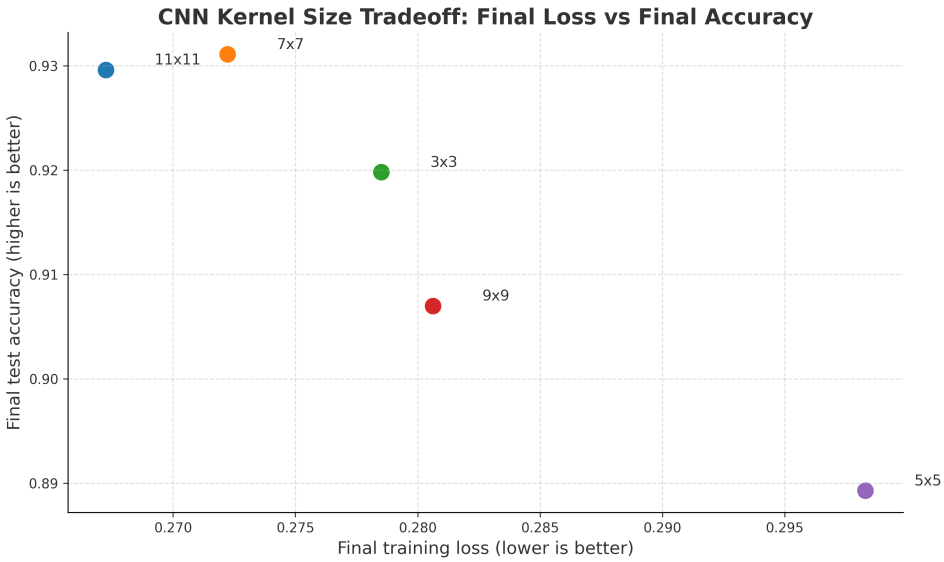

Figure 6: CNN kernel-size tradeoff between final training loss and final test accuracy.

Figure 7 shows that larger hidden size helps, but the gain after 128 neurons is small (from 0.9688 at 128 to 0.9705 at 256). I interpret this as a diminishing-returns point for the current training setup.

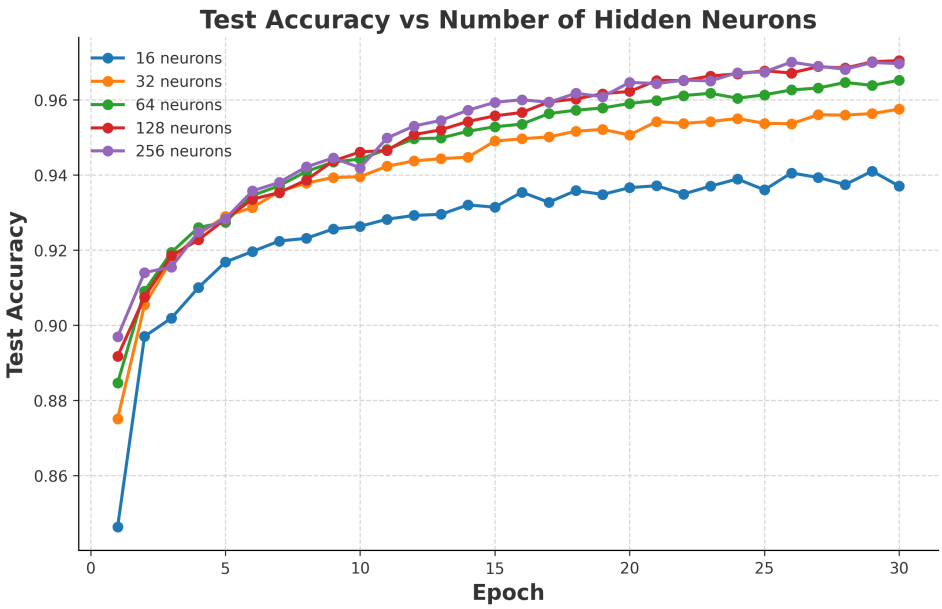

Figure 7: MLP hidden-size benchmark.

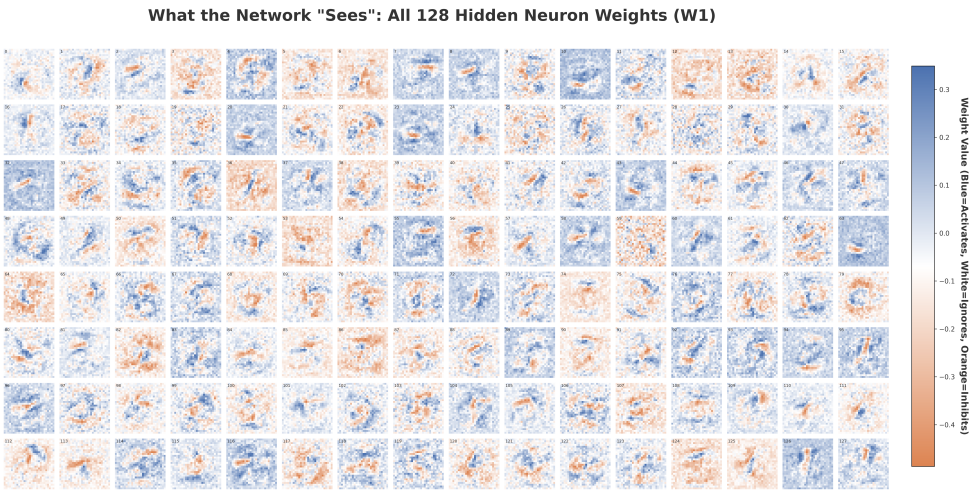

Figure 8: Learned first-layer MLP weights for 128 hidden neurons.

In Figure 8, different hidden units appear to focus on different stroke patterns. Some emphasize vertical or horizontal strokes, while others capture curved or crossing parts of digits. This suggests the hidden layer learns useful parts that the output layer later combines for final class decisions.

3.3 Qualitative Profiling and Confusion Analysis

Figure 9 and Figure 10 show a clear difference in error patterns. The MLP matrix is more concentrated on the diagonal, while the CNN matrix has more spread into off-diagonal cells

after 5 epochs. For example, MLP mistakes like $4 \to 9$ (15) and $5 \to 3$ (13) are limited, but CNN mistakes like $8 \to 3$ (132), $9 \to 3$ (122), and $0 \to 2$ (101) are much larger. I read this as CNN still being under-trained at this budget, especially on visually similar digits.

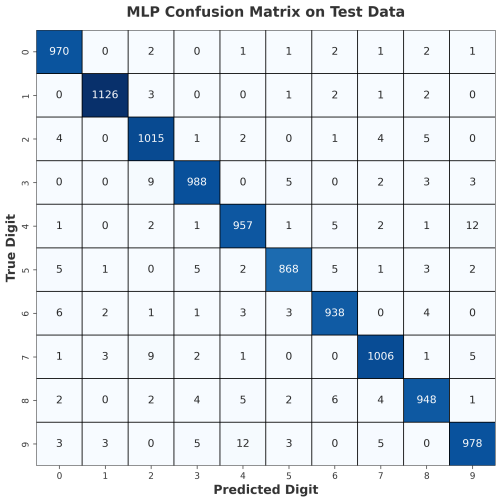

Figure 9: MLP confusion matrix.

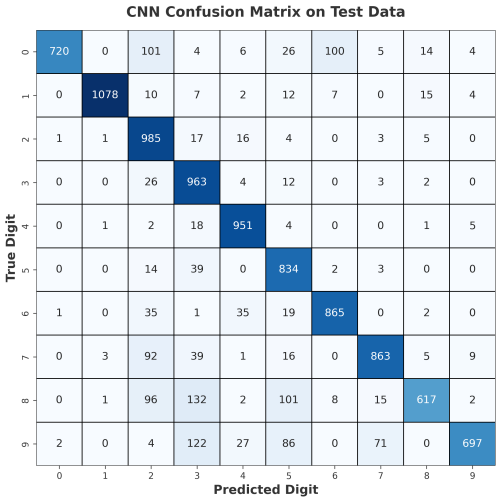

Figure 10: CNN confusion matrix.

A key limitation is that the training budgets are not equal (30 MLP epochs versus 5 CNN epochs, following assignment rules). Because of that, the performance gap should be read as a comparison under required settings, not as a final statement about each model's full potential.

4 Learning Outcomes

4.1 Tasks Accomplished

I completed manual implementations of both required models and their full training/evaluation pipelines in `MLP_template.py` and `CNN_template.py`. The main parts I finished were:

- Manual cross-entropy loss with numerical-stability fixes (max-shift Softmax and an ϵ term in the logarithm).
- Manual backpropagation for both models. For MLP, I computed Sigmoid-layer derivatives and updated parameters from those gradients. For CNN, I propagated gradients from the dense output layer back through flattening and then into the convolution kernel.
- Debugging CNN tensor shapes so each step matched correctly. I checked that convolution outputs, flattened vectors, and reshaped gradients all stayed compatible during both forward and backward passes.
- Targeted hyperparameter checks across the values I benchmarked, including MLP hidden size, MLP initialization scale, and CNN kernel size, to justify the final settings used in the templates.

4.2 Challenges and Debugging Realities

The main challenge was debugging the manual training logic without autograd to catch mistakes for me. I had to check the forward pass, the gradients, and the parameter updates

by hand, and the failed-guess training flowchart (Methodology, Figure 1) was one of the investigative steps I used to confirm that the correction signal was behaving as expected.

The CNN side was also constrained by the fixed 5-epoch limit in the project. That meant there was little room for slow convergence, so I had to keep the implementation simple, check the tensor shapes carefully, and use kernel-size benchmarking to choose a setting that learned reasonably well within the short training budget.

4.3 Lessons Learned

These experiments helped me understand how architecture choices and training budget affect results:

- The final MLP setting (hidden size 128, learning rate 0.5, 30 epochs) reached 0.9790 test accuracy. Hidden-size testing showed only small gains beyond 128, so that looks like a practical point for this setup.
- CNN test accuracy moved up and down across epochs ($0.9062 \rightarrow 0.8967 \rightarrow 0.9014 \rightarrow 0.8803 \rightarrow 0.9178$) even though loss kept decreasing. The main constraint was that the CNN was only allowed 5 epochs, so I treated the implementation as a short-training setup and used kernel-size benchmarking to make the most of that limited budget.
- Kernel-size testing selected 7×7 as the best option under the fixed 5-epoch budget, which supports using that value in `CNN_template.py`.
- Under the required budgets, the final gap between MLP (97.90%) and CNN (91.78%) is 6.12 percentage points. In this setup, the MLP benefited from both longer training and stronger dense-layer capacity.